Ontario Housing Price Prediction

Final Project

Josiah Baarbe,

Ram Chopra,

Kapil Patel, and

Ted Njoroge

COSC22001 – Machine Learning: Regression

April 20, 2026

Prof. Omar Al-Trad

Table of Contents

Section 1 - Data Collection 4

1. Overview 4

2. Data Sources at a Glance 4

a. Dataset Description 4

b. Dataset Visualization of Listings by Region 5

3. Acquisition Method 1 - Custom Scraper for zolo.ca 6

3.1 Motivation 6

3.2 Technology Stack 6

3.3 Code Structure (Zolo_Scraper_Code) 6

3.4 How the Scraper Works (End-to-End Flow) 7

3.5 Fields Captured by the Scraper 7

3.6 Example Invocation 8

3.7 Operational Notes and Safeguards 8

4. Acquisition Method 2 - RapidAPI Real-Estate Endpoint 9

4.1 Motivation 9

4.2 Acquisition Workflow 9

4.3 Fields Captured (RapidAPI_Dataset_Sold.csv) 9

4.4 Why This Second Source Is Valuable 10

5. Legal and Ethical Considerations 10

6 10

Section 2 - Data Preparation 12

1. Overview 12

1.1 Inputs and Outputs 12

1.2 Why the Cleaning Steps Are Split This Way 12

2. Step-by-Step Cleaning 13

3. Before / After Row Counts 15

4. Cleaned Dataset Schema 16

5. Remaining Missingness After Cleaning 16

6. How to Reproduce 16

7. Summary 17

Section 3 - Feature Selection and Engineering 18

Overview 18

3.1 – Purpose of Feature Engineering 18

3.2 – Starting Point for the Engineering Stage 18
3.3 – Initial Structured Features 18
3.4 – Price and Structure-Based Engineered Features 19
3.5 – House Age Encoding 19
3.6 – Additional Data Corrections Before Final Modeling 19
3.7 – Imputation of Engineered-Feature Nulls 19
3.8 – Consolidation of Rare Categories 19
3.9 – One-Hot Encoding of Categorical Variables 19
3.10 – Final Output of the Feature-Engineering Stage 20
Section 4 – F-Tests and Statistical Testing 21
ANOVA F-tests for categorical predictors 21
T-test for sold-above-asking behavior 21
Interpretation 21
Section 5 – Model Selection & Evaluation 22
Model Selection, Training, and Evaluation 22
Section 6 – Deployment & Application 23

Section 1 - Data Collection

1. Overview

The objective of this project is to build a regression model (Multiple Linear Regression enhanced with Ridge or Lasso regularization) that predicts the sold price of residential properties within the Greater Toronto Area. To train and evaluate such a model, a sufficiently large, feature-rich, and geographically representative dataset of real-estate listings across Durham Region, York Region, Peels Region, Halton Region and Toronto was required. Because no single public dataset satisfied these needs out of the box, the team combined two independent data acquisition strategies:

- A custom-built web scraper targeting zolo.ca, which produced two complementary datasets: one of recently sold listings and one of currently for-sale listings (the latter also carries Zolo's own price estimate, which serves as the primary benchmark defined in the project outline).
- A ready-to-use third-party real-estate API consumed through the RapidAPI marketplace, used to collect for-sale and for-sold listings from real-time data sources augmented from redfin.ca and realtor.ca, widening coverage beyond what the scraper alone could retrieve within the project timeline.

Using two independent sources improves data robustness: it reduces single-source bias, increases geographical and property-type diversity, and allows cross-validation of overlapping fields (price, MLS number, location, beds/baths) before the data is handed off to the preparation stage.

2. Data Sources at a Glance

a. Dataset Description

Dataset File	Origin and Purpose
Scraped_Dataset_Sold.csv	Recently sold Ontario listings scraped from zolo.ca. Provides the primary training signal - historical sold prices together with property attributes.
Scraped_Dataset_For_Sale.csv	Currently listed Ontario properties scraped from zolo.ca. Carries the zolo_estimate field, which is the benchmark used in the +/- 10% accuracy criterion defined in the project outline.
RapidAPI_Dataset_Sold.csv	Ontario sold-listings dataset collected through a real-estate API on RapidAPI. Supplementary source used to expand coverage and serve as an independent cross-check.
merged_dataset_v3.csv	Combines the union of both scraped and RapidAPI data sources based on MLS# Identifier, identifies inconsistencies between coordinates scraped, increases number of listings in Toronto region, and removes listings outside of the Greater Toronto Area.
reduced_dataset_v2.csv	Combines the intersection of both scraped and RapidAPI data sources based on MLS# Identifier, with listing_price included to calculate difference between listing and sold prices.
sample_v5.csv	Comprises for-sale listings currently on housing market across the Greater Toronto Area, with detailed analysis showing 3,027 listings in Durham Region, 5,096 listings in York Region, 11,001 listings in Toronto, 4,788 listings in Peel region, 2,639 listings in Halton and 1448 listings in Simcoe.

b. Dataset Visualization of Listings by Region

Initially, data was collected based on for-sale listings. The choropleths below show the number of listings collected based on for-sale listings; however, our model needs to estimate the actual value of a house rather than the listing price. This fundamental understanding shifted our approach to data collection and data gathering with extra effort put into ensuring sufficient number of listings between listing price and sold price.

Figure 1: The following choropleths show number of listings collected by region in GTA by data source

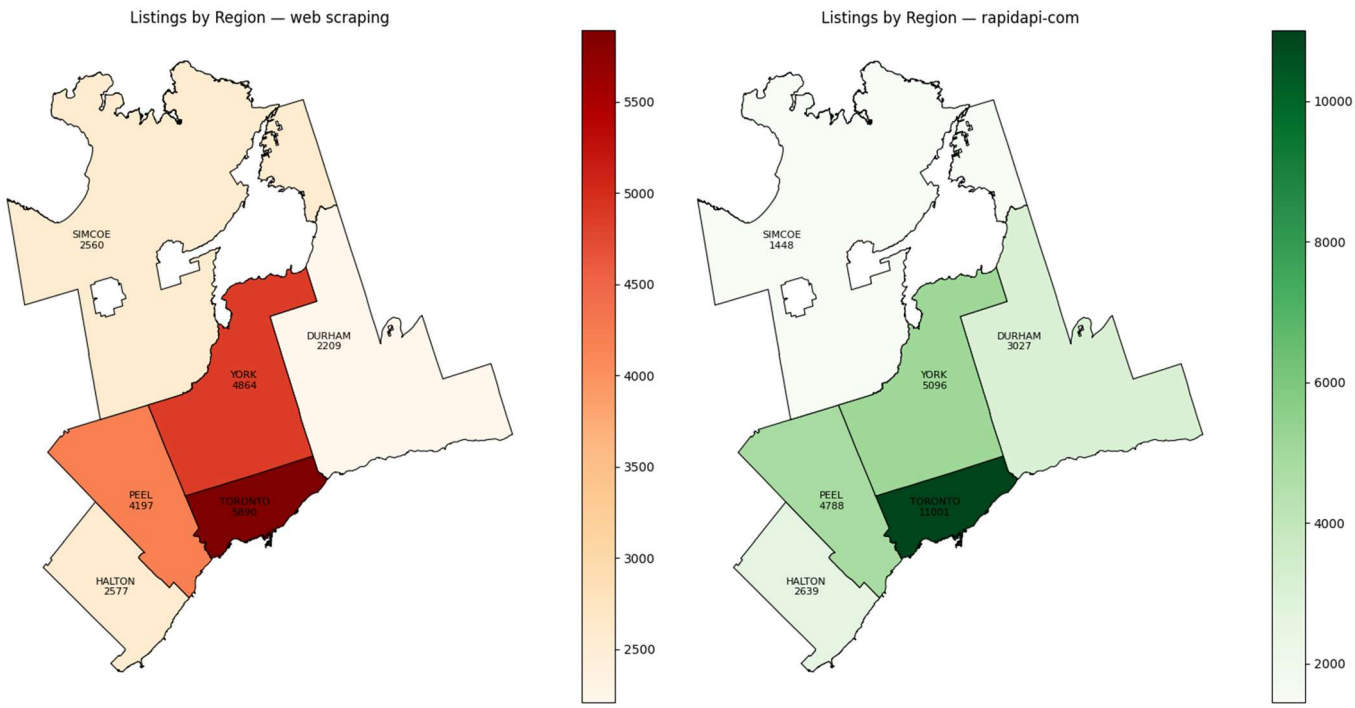
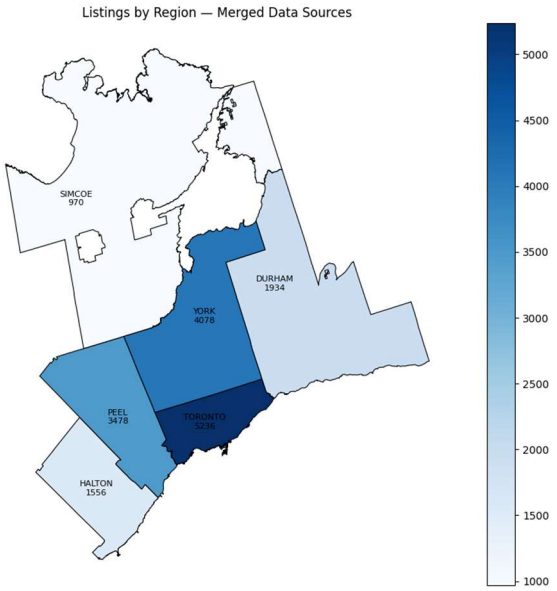


Figure 2: The following choropleths show number of listings collected by region after merging data sources



3. Acquisition Method 1 - Custom Scraper for zolo.ca

3.1 Motivation

zolo.ca aggregates MLS-backed Ontario real-estate listings and publishes a proprietary "Home Value" price estimate on for-sale pages. Because the project's first accuracy criterion requires comparing our model's prediction against Zolo's estimate, the most direct way to obtain consistently formatted, listing-level data together with that estimate was to scrape it directly from Zolo. No official public data export or dataset dump is offered by Zolo, so a custom scraper was built.

3.2 Technology Stack

Component	Choice / Rationale
Language	Python 3 (standard in the data-science workflow; easy CSV/JSONL output).
Browser automation	Playwright (Chromium). Zolo heavily client-side-renders listing details and uses anti-bot protections, so a real headless browser was required instead of plain HTTP requests.
HTML parsing	Dedicated parser module (parser.py) with label-based and regex-based extraction - resilient to Zolo's frequent DOM shuffles.
Persistence	Interim JSONL for raw scrapes (used for resume / debugging) and final CSV exports reduced to the fields used for modelling.
Session handling	Persistent Playwright user-data directory. Sold-price access on Zolo requires a signed-in session, so a one-time manual login is performed and the authenticated profile is reused across subsequent headless runs.

3.3 Code Structure (Zolo_Scraper_Code)

The scraper lives under Section_wise_Code_Files/Section-1_Data_Collection/Zolo_Scraper_Code/ in the project repository. The Python package is organised under src/zolo_scraper/ with clearly separated responsibilities:

Module	Responsibility
cli.py	Command-line entry point for single-city scrapes (accepts --city, --statuses, --max-pages, login flags, output paths).
harvest.py	Province-wide harvest driver. Expands Ontario / Canada-wide targets into city-level crawls, supports parallel sold/for-sale workers, and uses cloned browser profiles to avoid Chrome profile-lock conflicts.
scraper.py	Core scraping engine: seed discovery from listing feeds, listing-card parsing, detail-page navigation, deduplication, resume logic, and incremental CSV writing.
parser.py	All HTML-to-structured-data extraction: summary labels (Type, Style, Size, Lot Size, Age, Taxes, Walk Score, MLS #, ...), Sold/Price History blocks, Home Value section, city-guide demographic fields, and coordinates.

exporter.py	Normalisation step. Converts rich JSONL rows into the reduced 23-column modelling CSVs; expands completed Sold History blocks into one CSV row per finalised sale event.
config.py	Typed scrape configuration (feeds, statuses, output paths, rate limits).
audit.py	Row-count / resume baselines and live progress reporting.
tests/	Unit tests covering harvest logic and parser edge cases.

3.4 How the Scraper Works (End-to-End Flow)

A typical Ontario run proceeds through the following stages:

- Seed discovery: city slugs for the Greater Toronto Area (GTA) are expanded first, then the remaining Ontario cities are enumerated in discovered order. Zolo's separate Toronto sub-area slugs (north-york-real-estate, scarborough-real-estate, etobicoke-real-estate, east-york-real-estate, york-real-estate) are included so Toronto inventory is not missed.
- Listing-feed crawl: for each city, the scraper paginates /<city>/sold and /<city> up to --max-pages, collecting listing cards. Direct detail links exposed on the cards are preferred; slug reconstruction is used only as a fallback.
- Detail-page scrape: Playwright opens each listing in Chromium, waits for client-side rendering, and the parser extracts the full field set (see section 3.5).
- Sold History expansion: when a detail page exposes a completed Sold History block, each finalised sale event is emitted as its own CSV row (this is why, for example, a single address such as 20 Bridesburg Drive can appear multiple times in the sold dataset, one row per historical sale). Blocks containing only Listed or Removed events are ignored.
- Home Value capture: on for-sale pages, the Home Value section is parsed into the zolo_estimate column - this is the field that drives the project's +/-10% benchmark evaluation.
- Deduplication & resume: an interim JSONL file accumulates every raw scrape. A stable listing identity (MLS-based when available, otherwise address/city + canonical URL) prevents duplicates and allows interrupted runs to resume without reprocessing or dropping rows.
- Export: after each listing is stored, the exporter converts the JSONL into two reduced CSVs - one for sold, one for for-sale - both sharing the same 23-column schema used for modelling.
- Parallel harvest: sold and for-sale harvests can be launched as concurrent child processes (--parallel-statuses) with cloned Playwright profiles, roughly halving wall-clock time for a full Ontario sweep.

3.5 Fields Captured by the Scraper

Each row in Scraped_Dataset_Sold.csv and Scraped_Dataset_For_Sale.csv shares the same 23-column schema, summarised below:

Column	Description / Source
address	Street address from the listing card and detail page.
city	City resolved from structured fields, locality metadata, and the URL slug (hardened against false positives from comma-containing price or date lines).
listing_price	List price. Comes from the completed sold-history block when present, otherwise from the current page-level listing field.

sold_price	Final sale price (only reliably populated when the session is signed in). Sold rows without a numeric sold_price are dropped by default.
listing_date / sale_date	Listed and sold dates - from the sold-history block when available, otherwise from page-level fields.
bedrooms / bathrooms	Bed and bath counts from the detail page summary.
square_feet	Preserved verbatim as Zolo displays it (e.g. '600-699 sqft') rather than being forced to a midpoint - this preserves information for later feature engineering.
number_of_storeys	From the Stories field when present, with fallback inference from the Style string (e.g. '2-Storey').
latitude / longitude	Geocoordinates from the detail-page map metadata.
property_type	Zolo property-type string (e.g. Condo Apartment, Detached, Duplex, ...).
house_age / lot_size / taxes / walk_score / parking_spaces	Summary-section fields from the detail page.
zolo_estimate	Zolo's own price estimate from the Home Value block, primarily present on for-sale pages. This is the benchmark against which the +/-10% accuracy criterion is evaluated.
mls_number	MLS number - the historical MLS from the sold-history block when available, otherwise the current listing's MLS.
source	Originating board/source (e.g. TRREB).
median_income / population	City-level demographics pulled from Zolo's city guide pages and joined onto every listing in that city.

3.6 Example Invocation

The exact commands used to produce the Ontario datasets (run from Section_wise_Code_Files/Section-1_Data_Collection/Zolo_Scraper_Code/ with the Python virtual environment activated):

Sold-only harvest:

PYTHONPATH=src python -m zolo_scraper.harvest --regions ontario-real-estate --statuses sold --clone-user-data-dir --user-data-dir .playwright-profile/zolo --headless --sold-output-csv output/final/ontario-real-estate_sold_only.csv --jsonl-output output/interim/ontario_listings_sold.jsonl

For-sale harvest (ran in a separate terminal):

PYTHONPATH=src python -m zolo_scraper.harvest --regions ontario-real-estate --statuses for_sale --clone-user-data-dir --user-data-dir .playwright-profile/zolo --headless --for-sale-output-csv output/final/ontario-real-estate_for_sale_only.csv --jsonl-output output/interim/ontario_listings_for_sale.jsonl

The final CSVs produced by these runs are the files named Scraped_Dataset_Sold.csv and Scraped_Dataset_For_Sale.csv, which live under Collected_Datasets/ in the project repository.

3.7 Operational Notes and Safeguards

- Rentals are explicitly excluded; only sale listings are kept.

- Ontario harvest is GTA-first by design, which matches the project's regional focus and keeps the dataset dense in the markets where Zolo's coverage is strongest.
- Resume logic tracks source/canonical URL aliases plus a stable listing identity (MLS-first, address/city + canonical URL fallback), so an interrupted run can be restarted without repeating rows or silently missing listings.
- Because sold-price access on Zolo requires authentication, a one-time manual login is performed with --manual-login; the authenticated Playwright profile is then reused across subsequent headless runs.
- Rate-limiting and user-agent spoofing are applied by Playwright to keep the crawl within normal browsing behaviour.

4. Acquisition Method 2 - RapidAPI Real-Estate Endpoint

4.1 Motivation

While the custom scraper provided rich, benchmark-aligned data from Zolo, web scraping is inherently rate-limited and subject to layout changes. For example, the coordinate values in the html header differed from the coordinate values of the actual listing; however, the web scraper would only include the html header values if present instead of checking consistency between the html header values and coordinate values of the listing. To correct errors in data collection and increase the total number of sold listings within the past 365 days, the team used a real-time, ready-to-use, real-estate API hosted on RapidAPI marketplace. RapidAPI exposes third-party APIs through a unified authentication and billing layer, which made it straightforward to pull sold-listings data from the Greater Toronto Area without building any additional scraping infrastructure.

4.2 Acquisition Workflow

- Subscribe to the real-estate endpoint on RapidAPI and obtain an API key.
- Iterate over Ontario city queries via the endpoint's search/listing routes, requesting sold listings.
- Paginate through responses and persist each listing as a JSON record.
- Flatten the nested JSON response (homeData.* paths) into a tabular CSV - this is the reason the column names are prefixed with homeData_, keeping full traceability back to the raw API schema.
- Derive helper columns used downstream in modelling: sold_price (extracted from the price-info object), sold_date_dt (parsed datetime), and days_since_sold (recency feature).

4.3 Fields Captured (RapidAPI_Dataset_Sold.csv)

The API returns a more verbose schema than the scraper. The most relevant columns for modelling are:

Column	Description
homeData_propertyId / homeData_listingId / homeData_mlsId	Unique identifiers from the API; used for deduplication and cross-matching against the Zolo scrape via MLS number.
homeData_url	Relative detail-page path, which also encodes the province (e.g. /on/...) and city slug.
homeData_propertyType	Property-type code.
homeData_beds / homeData_baths / homeData_bathInfo_computedTotalBaths	Bedroom and bathroom counts.
homeData_priceInfo_amount / homeData_priceInfo_homePrice_int64Value	Listed/asking price information.
homeData_sqftInfo_displayLevel	Flag indicating whether square-footage was disclosed.

homeData_lotSize_amount / homeData_lotSize_displayLevel	Lot size and its disclosure level.
homeData_yearBuilt_displayLevel	Year-built disclosure flag.
homeData_hoaDues_amount	HOA / condo fees.
homeData_addressInfo_centroid_centroid_latitude / _longitude	Geocoordinates.
homeData_addressInfo_formattedStreetLine / _city / _zip / _location / _unitNumber / _countryCode	Structured address components.
homeData_daysOnMarket_listingAddedDate	Date the listing was added - used for recency features.
homeData_sashes	Listing sashes (e.g. Sold badge with lastSaleDate, Video Tour); lastSaleDate is parsed into sold_date_dt.
sold_price / sold_date_dt / days_since_sold	Derived modelling columns: the actual sale price, the parsed sale date as a timezone-aware datetime, and the number of days since that sale - the last one is a useful recency feature for regression.

4.4 Why This Second Source Is Valuable

- Independence: the RapidAPI source does not share the Zolo scraping pipeline, so errors in one pipeline do not propagate to the other.
- Coverage: it includes ~~Ontario~~ markets within the Greater Toronto Area ~~and sub-regions~~ that were below the Zolo scraper's priority slugs, broadening the geographic distribution of training data.
- Cross-validation: overlapping listings (matched by MLS number or address + sold-date) provide a sanity check on prices, coordinates, and property attributes before modelling.

5. Legal and Ethical Considerations

- All scraping was performed at modest request rates with realistic browsing patterns to avoid placing undue load on zolo.ca.
- Only publicly displayed listing information was collected; no user accounts, personal data, or private contact details were harvested.
- The RapidAPI endpoint was used strictly within the plan's subscription limits and terms of service.
- The collected datasets are used solely for academic coursework and are not redistributed outside of this project.

6. Summary

In total, three Ontario real-estate datasets were assembled from two independent acquisition pipelines. Together they provide the property-level features, the ground-truth sold prices, and the Zolo benchmark estimates required to train and evaluate the regularised multiple linear regression model in the sections that follow:

- Scraped_Dataset_Sold.csv - historical sold listings from zolo.ca with full property attributes and sold price (primary training signal).
- Scraped_Dataset_For_Sale.csv - currently listed properties from zolo.ca, carrying zolo_estimate for the +/- 10% benchmark comparison.

- RapidAPI_Dataset_Sold.csv - Ontario sold listings obtained via a RapidAPI real-estate endpoint, expanding coverage and serving as an independent cross-check.

Section 2 - Data Preparation

1. Overview

In Section 1 we datasets from two different sources with an emphasis on ensuring data robustness to prevent noise and data leakage. Raw real-estate data is almost never ready to be fed into a regression model directly: prices contain data-entry errors, text fields like `square_feet` come in as ranges, property types have casing inconsistencies, and some columns are almost entirely empty. Section 2 documents every cleaning and transformation step we applied to turn those raw CSVs into the cleaned datasets that the model in Section 5 will train on.

A guiding principle for this section: we kept every step as simple as possible. All cleaning is done with pandas and numpy, using basic operations that were covered in the course - string parsing, boolean filtering, value replacement, date parsing, and one regular expression per text column. We deliberately do NOT perform imputation, one-hot encoding, or feature scaling at this stage. Those steps are deferred to the scikit-learn Pipeline in the model-training step so that they are fit only on the training split (preventing any data leakage from the test split).

1.1 Inputs and Outputs

Stage	File	Purpose
Input	merged_dataset.csv	Dataset comprising of 32,393 listings, and establishes the starting dataset with initial view of shape and missingness
~~**Input**~~	~~Scraped_Dataset_For_Sale.csv~~	~~Raw scraped Zolo for-sale listings (27,500 rows) - benchmark source (contains zolo_estimate).~~
~~**Input**~~	~~RapidAPI_Dataset_Sold.csv~~	~~Raw RapidAPI sold listings (32,393 rows) - secondary cross-check source.~~
Output	cleaned_dataset.csv	Preserves a reusable preprocessing output for later modeling
~~**Output**~~	~~data_cleaned/Cleaned_Dataset_For_Sale.csv~~	~~Cleaned benchmark dataset, one row per listing (23,814 rows).~~
~~**Output**~~	~~data_cleaned/Cleaned_Dataset_RapidAPI.csv~~	~~Cleaned cross-check dataset, shared feature subset (32,376 rows).~~

1.2 Why the Cleaning Steps Are Split This Way

Cleaning operations fall into two categories: row-independent transformations (safe to apply before the train/test split) and parameter-fit transformations (must be learned only from the training data). We separate them accordingly:

Where	What we do
Section 2 (this stage, applied to every row)	Column drops, residential-type filtering, price/tax sanity filters, text-to-number parsing (square_feet, lot_size, house_age), categorical label normalisation, date parsing, missingness flag for house_age, duplicate removal.

Section 5 (inside the sklearn Pipeline, fit on train only)	Median imputation of remaining NaNs, most-frequent imputation for the categoricals, OneHotEncoder for property_type and city, StandardScaler for the numeric features, log-transform of the target.

2. Step-by-Step Cleaning

2.1 – Consolidating and Standardizing Core Fields

2.1 Consolidating and standardising core fields

First stage of scrubbing the merged dataset began with consolidating duplicate target columns into a single `sold_price` field using `sold_price_x` and `sold_price_y`. Key columns were then renamed to ensure consistency and easy understanding, including transforming `square_feet` to `square_feet_num` and `lot_size` to `lot_size_sqft`. Making these changes ensured our merged dataset had a single target variable and increase understanding using clear field names before further preprocessing. Some of the incomplete fields included:

Field	Missing Data (Percentage)
`zolo_estimate`	100.0%
`walk_score`	100.0%
`lot_size`	68.1%
`number_of_storeys`	63.4%
`Bedrooms`	62.9%
`Bathrooms`	62.8%
`house_age`	62.8%

2.2 – Creating initial Structured Features

The initial row-wise features were created directly from existing columns. For example, `listing_year` and `listing_month` were extracted from `listing_date`. Similarly, `bath_bed_ratio` was computed based on the ratio between bathroom and bedroom counts. Extracting critical information into new row-wise features provided an initial structured representation with regards to week of the month and type of housing unit, resulting in a dataset with shape of 32,393 rows and 68 columns.

2.3 – Recovering Missing Values from Parallel Merged Fields

The merged dataset comprised of several overlapping fields from multiple sources, as well as several missing values which enabled easier recovery based on corresponding `home_data_*` columns. For example, missing values in bedrooms, bathrooms, city, listing_price, latitude, and longitude from the scraped dataset were filled from their parallel homeData versions. However, further inspection revealed inconsistencies between the scraped dataset and the dataset retrieved from RapidAPI marketplace. Closer inspection revealed the scraped dataset comprised of erroneous longitudinal/latitude coordinates which were gathered from the html header instead of the correct values corresponding to the listing. Combining both datasets ensures data consistency but also requires more effort with extrapolating and scrubbing to ensure consistency across both sources. Our

processed dataset removes coordinates and opts for city-based hot-encoding which is discussed in further detail later in this report. Lastly, `lot_size_sqft` values were supplemented using `homeData_lotSize_amount.` During this step, over 20,299 rows were eliminated with missing count greater than 24 which implies uneven balance between datasets when merging. This reduced the number of missing values significantly while there were other fields with still considerable high percentage of missing values.

Figure 2.3a: Decrease in missing values after removing 20,299 rows

Field	Missing Data (Percentage)
bedrooms	0.4%
bathrooms	0.2%
city	0.0%
listing_price	0.0%
latitude	0.0%
longitude	0.0%
sold_price	0.0%

Figure 2.3b: Removing highly sparse rows did not significantly reduce missing values in all fields

Field	Missing Data (Percentage)
lot_size_sqft	68.1%
parking_spaces	62.9%
house_age	62.8%
taxes	62.7%
square_feet_num	62.7%
median_income	62.7%
property_type	62.7%

2.4 – Removing Redundant Source Columns

After recovery was complete, redundant homeData_* columns were removed to reduce collinearity between features. Additional non-modeling identifiers and metadata, such as `source, mls_number, address`, and date fields no longer needed in raw form, were also dropped. Removing duplicate columns simplifies the schema and limits leakage from raw source fields.

2.5 – Keeping Records with Essential Info

Any records missing core fields (e.g., `sold_price`, `listing_price`, etc) required for downstream analysis were removed. In the notebook workflow, this included rows missing values such as: `sold_price`, `listing_price`, bedrooms, bathrooms, `city`, `property_type.` This step ensured our dataset contained, at the very least, the minimum information needed for modeling and feature engineering.

2.6 – Parsing and Encoding

Qualitative fields were converted into numeric form so they could be used in analysis and modeling. For example, `square_feet_num` was parsed from square-footage text using a custom parser. Similarly, `lot_size_sqft` was encoded into numeric format. Transforming categorical and qualitative fields into size-related variables enabled usable numeric predictors which increases the accuracy of the overall model.

2.7 – Handling Categorical Missingness

The categorical field `house_age` had missing values which were filled with the label `Unknown`. This preserved rows that would otherwise be lost while retaining information that the age value had originally been unavailable.

Figure 2.6: Number of rows per unique value for field house_age

Field: HOUSE_AGE	Number of Rows
Unknown	19,936
No Data	7,415
16-30	1,075
31-50	902
0-5 years	683
6-15	602
51-99	542
New	336
6-10	137
11-15	98
100+	63

3. Before / After Row Counts

The table below shows how many rows survived each stage of the pipeline. Overall we retained about 97% of the raw sold listings, which is a very healthy retention rate - we did not have to throw away large portions of the data just to make it clean.

Stage	Scraped Sold (primary)	Scraped For-Sale (benchmark)
Raw rows loaded	34,834	27,500
After residential-type filter	34,677	25,882
After sanity filters	33,905	23,967
After deduplication	33,905 (final)	23,814 (final)
Retention vs raw	97.3%	86.6%

For the RapidAPI cross-check dataset we applied only the shared-subset cleanup (column renaming to our canonical schema, price sanity filter, city title-casing, date parsing, duplicate removal). 32,393 raw rows became 32,376 cleaned rows (99.9% retention).

4. Cleaned Dataset Schema

Every row in Cleaned_Dataset_Sold.csv has the columns listed below. The same schema is used for Cleaned_Dataset_For_Sale.csv with one addition (zolo_estimate) and a reduced subset is used for the RapidAPI cleaned file.

Column	Type / Role
mls_number, address, source	Identifiers. Kept for traceability only - not passed to the model.
sold_price	Target variable (CAD).
listing_price	Numeric feature.
bedrooms, bathrooms, number_of_storeys, parking_spaces	Numeric features (integer counts).
square_feet_num, lot_size_num, house_age_num	Numeric features derived from the parsed string columns.
taxes, median_income, population	Numeric features.
latitude, longitude	Numeric features capturing spatial location.
days_on_market, sale_year, sale_month	Date-derived numeric features.
property_type, city	Categorical features (to be one-hot encoded in the model step).
house_age_missing	Binary flag indicating the original house_age was 'No Data'.
listing_date, sale_date	Parsed datetime columns, kept for reference.
zolo_estimate (for-sale file only)	Benchmark used in the +/-10% evaluation criterion.

5. Remaining Missingness After Cleaning

After Section 2 the main sold dataset has missing values only in the columns listed below. All of these are handled in Section 5 using a SimpleImputer fit only on the training split.

Column	% NaN	Plan for Section 5 (inside sklearn Pipeline)
house_age_num	~65%	Median imputation. The house_age_missing flag preserves the missingness signal.
lot_size_num	~22%	Median imputation. Most NaNs are condos/apartments where lot size is not meaningful.
number_of_storeys	~1%	Median imputation.
days_on_market	~0.1%	Median imputation.
square_feet_num	~0.04%	Median imputation by property_type group.

6. How to Reproduce

All of the cleaning logic lives in src/data_preparation.py. The notebook notebooks/02_data_preparation.ipynb walks through each step interactively using the same helper functions. To regenerate the cleaned CSV files from the raw files, run from the project root:

```
python src/data_preparation.py
```

This will read the three raw CSVs from the project root, apply every step described above, and write the cleaned files to data_cleaned/.

7. Summary

At the end of Section 2 we have three cleaned, consistently-typed, duplicate-free datasets ready for feature selection in Section 3 and for regression modelling in Section 5. We intentionally kept imputation, scaling and encoding out of this stage so they can be fit inside the training pipeline without leaking information from the test split.

Section 3 - Feature Selection and Engineering

Overview

In Section 2 we produced merged_dataset.csv, which contains over 33,000 listings across the Greater Toronto Area region; with fields comprising of sold listings with parsed numeric fields, normalised categoricals, and a few rows dropped for failing basic sanity checks. The job of Section 3 is to turn that tidy dataset into a feature matrix that a regularized linear regression model can actually learn from, and to decide which of those features are worth keeping.

We split the work into two halves. The first half is feature engineering: log-transforming the skewed columns, adding a handful of simple derived features, one-hot encoding the categoricals, and standardising the numerics so that Lasso treats every feature on the same scale. The second half is feature selection, where we run the three methods covered in our course (Sequential Forward Selection, Sequential Backward Elimination with OLS p-values, and Lasso L1) on the engineered feature matrix, compare what they pick, and take a majority vote.

Throughout we deliberately stay on the simple side. The course covered basic regression and regularisation, so we use only techniques that fit inside that scope. Where a choice had to be made between a fancier approach and a plain one, we took the plain one and explain the trade-off.

3.1 – Purpose of Feature Engineering

After the initial cleaning and recovery steps, the notebook moved from basic preprocessing to feature engineering so the merged housing dataset could be converted into a model-ready matrix. In the updated workflow, this stage focused on transforming cleaned housing attributes into more informative numerical predictors, encoding categorical variables, and ensuring that the final dataset could be used directly for modeling. This is different from the earlier draft, which emphasized a pipeline built around log transforms and a 41-feature matrix; the current notebook instead uses a broader feature-construction workflow ending in a much wider engineered dataset.

3.2 – Starting Point for the Engineering Stage

Feature engineering began after the dataset had already gone through column consolidation, missing-value recovery from parallel homeData_* fields, removal of redundant columns, essential-field filtering, numeric parsing, and imputation. The preprocessing table shows that the notebook first created a unified sold_price, standardized key field names such as square_feet_num and lot_size_sqft, profiled row-level sparsity, recovered missing values from merged-source columns, imputed remaining numeric and categorical gaps, and removed implausible records before beginning the main feature-engineering steps.

3.3 – Initial Structured Features

Before the main engineered predictors were added, the notebook created early structured features from existing columns. Specifically, it extracted listing_year and listing_month from listing_date and computed an initial bath_bed_ratio. These features preserved useful timing and housing-composition information in a format that could later be incorporated into regression modeling.

3.4 – Price and Structure-Based Engineered Features

The core feature-engineering stage added several pricing and structural variables intended to capture housing value relationships more directly than the raw columns alone. According to the notebook-derived preprocessing table, the engineered variables included price_per_sqft, price_diff_pct, sold_over_asking, beds_plus_baths, taxes_per_sqft, and a refreshed bath_bed_ratio. These variables were designed to summarize how listing price, sold price, taxes, and housing characteristics interact, thereby improving the explanatory power of the final model matrix.

3.5 – House Age Encoding

The notebook also transformed house_age into a model-usable ordinal field called encoded_house_age. Before encoding, missing house_age values were filled with Unknown, so that rows would not be discarded unnecessarily and the absence of age information would still be represented consistently. After the encoded version was created, the original raw house_age field was removed to avoid duplicate representation of the same concept.

3.6 – Additional Data Corrections Before Final Modeling

Two final rule-based corrections were applied during this stage to improve data validity. First, rows with listing_price below 1000 were removed as implausible observations. Second, bedroom counts equal to zero were replaced with one so that invalid room counts would not distort downstream ratios and engineered housing features. These corrections were applied before the final categorical encoding and export steps.

3.7 – Imputation of Engineered-Feature Nulls

Because some derived variables introduced new missing values, the notebook explicitly imputed the engineered fields price_per_sqft and taxes_per_sqft using median values. This step ensured that the addition of new pricing features did not reintroduce incompleteness into the dataset immediately before modeling. The preprocessing table identifies this as a separate step following feature creation, which shows that the notebook treated engineered-feature null handling as part of the formal workflow rather than as an implicit cleanup step.

3.8 – Consolidation of Rare Categories

To reduce sparsity in the categorical design matrix, the notebook consolidated infrequent property_type levels into a single category labeled others. The preprocessing table states that only the most frequent property_type categories were retained explicitly, while the rest were grouped together. This step reduced fragmentation in the categorical space and made the subsequent one-hot encoding more stable.

3.9 – One-Hot Encoding of Categorical Variables

Once rare categories had been consolidated, the categorical variables property_type and city were converted into numeric form using pd.get_dummies(..., drop_first=True). This transformed both categorical columns into model-ready indicator variables suitable for linear regression and related algorithms. The earlier draft described a different categorical engineering process based on training-fold grouping and a smaller encoded matrix, but the updated notebook workflow documents direct one-hot encoding as the actual implemented step.

3.10 – Final Output of the Feature-Engineering Stage

The final step of this stage was to save the processed DataFrame to cleaned_dataset.csv, preserving a reusable engineered dataset for later modeling, feature selection, and evaluation. This makes the feature-engineering stage the bridge between raw merged data preparation and the downstream predictive modeling stages of the project.

Section 4 – F-Tests and Statistical Testing

To assess whether selected engineered and categorical features were statistically associated with housing sale prices, the notebook included both **ANOVA F-tests** and an **independent two-sample t-test**. These tests were used as supporting evidence during exploratory analysis before final model fitting.

ANOVA F-tests for categorical predictors

A one-way ANOVA was applied to the categorical variables `city` and `property_type` to test whether mean `sold_price` differed across their categories. For `city`, the test used 20 groups and produced an **F-statistic** of 50.79 with a **p-value** < 0.000. For `property_type`, the test used 22 groups and produced an **F-statistic** of 182.32 with a **p-value** effectively equal to 0. Since both p-values were far below 0.05, the we concluded that both variables were statistically significant predictors of sold price and should be retained for later one-hot encoding.

```
---ANOVA Feature Significane-----
Feature: city  (20 groups)
 F-Statistic:  50.79
 P-Value:     7.3198e-185
 Result: Statistically significant keep for One-hot encoding
--------------------------------------------
Feature: property_type  (22 groups)
 F-Statistic:  182.32
 P-Value:     0.0000e+00
 Result: Statistically significant keep for One-hot encoding
--------------------------------------------
```

T-test for sold-above-asking behavior

The notebook also performed a Welch two-sample t-test comparing sold prices for listings that sold above asking versus those that sold at or below asking. The test produced a t-statistic of 3.6353 and a p-value < 0.0003 (p-value = 2.8351×10^{-4}), indicating a statistically significant difference in mean sold price between the two groups.

Listings sold above asking	Mean sold price
2,299	$1,194,101

Listings sold at or below asking	Mean sold price
9,902	$1,036,658

Interpretation

Overall, these statistical tests supported the inclusion of important categorical and market-behavior variables in the modeling workflow. The ANOVA results showed that both **location (city)** and **housing category (property_type)** are strongly associated with sale price, while the t-test showed that the **sold-over-asking pattern** captures a meaningful difference in pricing outcomes.

Section 5 – Model Selection & Evaluation

Model Selection, Training, and Evaluation

The final modeling dataset contained 12,017 observations and 35 columns, consisting of 34 predictor variables and 1 target variable (`sold_price`). The data were split into an 80/20 train-test partition, producing 9,613 training rows and 2,404 test rows, with `random_state=42` used to ensure reproducibility.
Three baseline linear models were evaluated on the full feature set: 1) Ordinary Least Squares (OLS), 2) Ridge, and 3) Lasso. All three performed almost identically, indicating that the engineered features already provided a highly predictive representation of the housing data. OLS and Ridge each achieved a MAPE of 3.85%, R^2 of 0.9867, 92.7% of predictions within 10% of the true sold price, and 98.0% within 15%. Lasso performed similarly, with a MAPE of 3.84%, R^2 of 0.9867, 92.6% within 10%, and 97.9% within 15%.

To assess stability, the notebook ran 5-fold cross-validation using OLS on the full feature set. The cross-validated R^2 scores ranged from 0.9855 to 0.9867, with a mean of 0.9860 and a standard deviation of 0.0004. This very small variation suggests that model performance was highly consistent across folds and that the selected predictors generalize well rather than overfitting to one particular split.

Feature selection was then carried out using Sequential Forward Selection (SFS) and Backward Elimination Selection (BES). SFS selected 17 features, while BES also selected 17 features, with substantial overlap between the two methods. The final model used the union of both selected feature sets, producing a more compact subset of variables while preserving predictive performance. The retained predictors included structural variables such as listing_price and bathrooms, along with several encoded property type and city indicators.

The final OLS model, trained only on the selected features, achieved essentially the same performance as the full-feature baselines: MAPE = 3.84%, R^2 = 0.9867, 92.7% within 10%, and 97.8% within 15%. This shows that feature selection reduced dimensionality without materially reducing accuracy. Among the final model coefficients, listing_price had by far the largest positive effect (523,641.25), followed by bathrooms (18,256.96) and city_York (12,207.45), indicating that asking price, housing size/amenities, and location were the strongest predictors of sold price in the final model.

Section 6 – Deployment & Application

Deploying the model, we used the streamlit python packaging to show an MVP that forecasts estimated value price of a housing unit based on city, property type, listing price, and number of bathrooms. The deployed model generalizes well in comparison to other housing predictor models when compared to *zolo.ca* and *redfin.ca*.

Figure 6.1: Screenshot of the GTA Home Price Predictor Application Using Streamlit

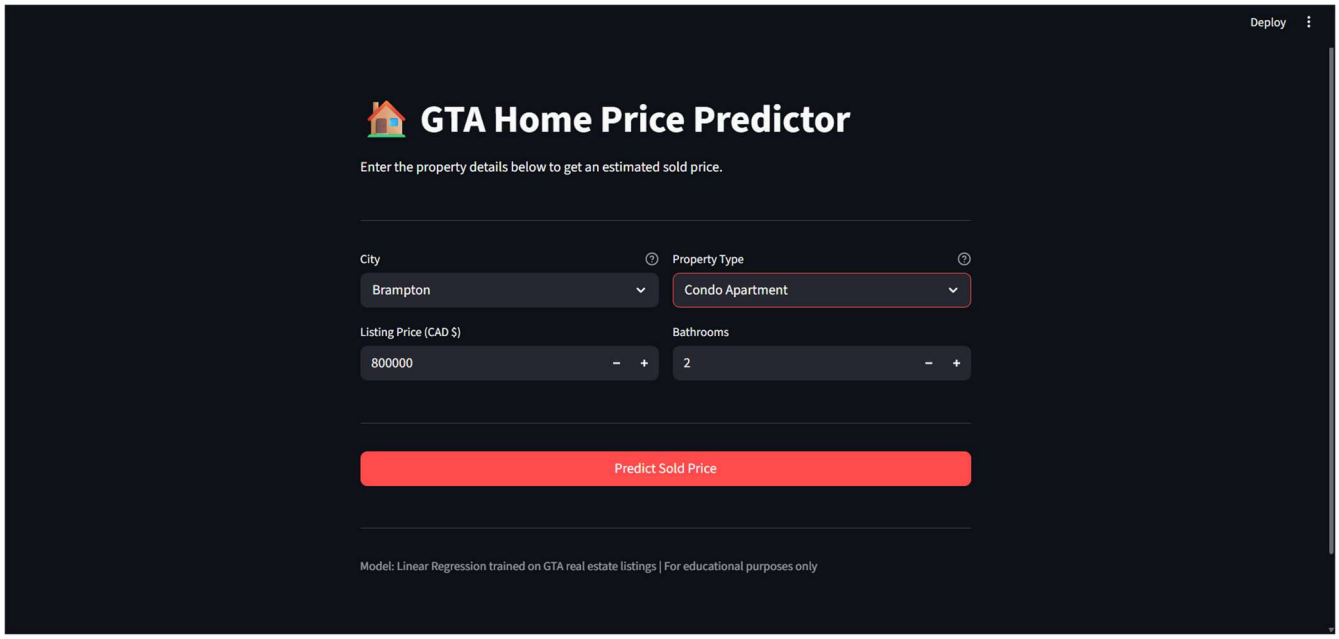
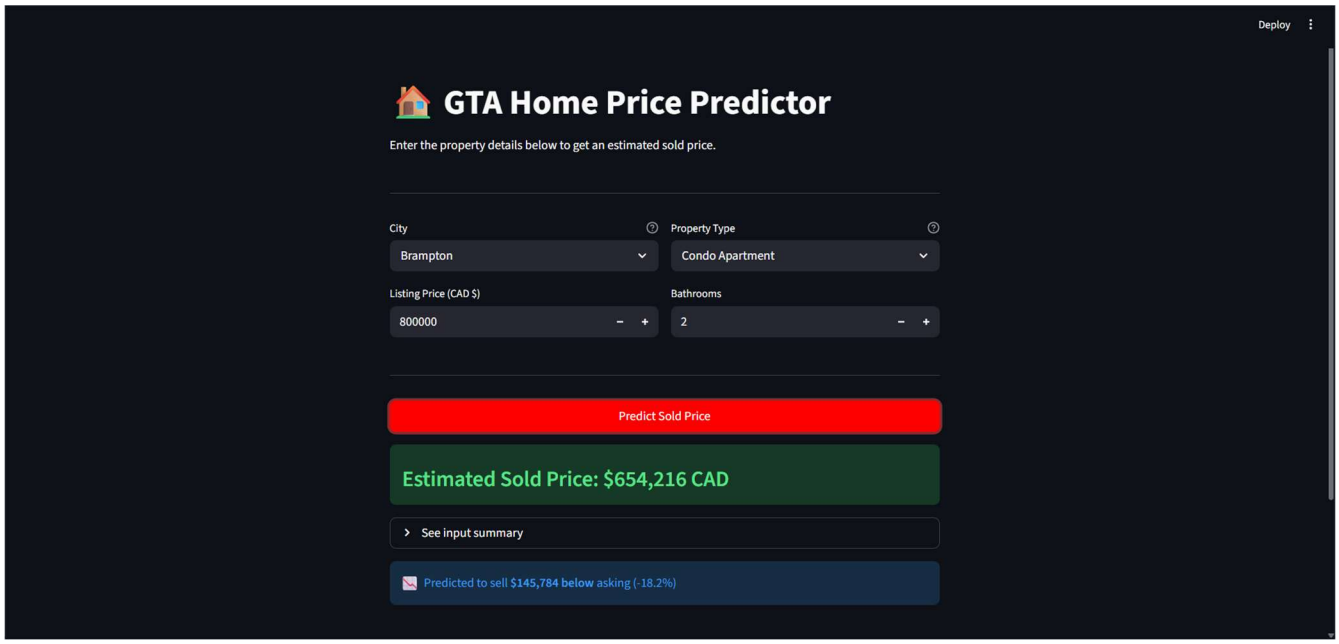



Figure 6.2: Screenshot of the GTA Home Price Predictor Application with Sample Forecast